



**Universidad autónoma de baja california**

**Ingeniería en computación**

**Proyecto de carrera**

**LAB-TALLER 9: MQTT**

**Aguilar Noriega Leocundo**

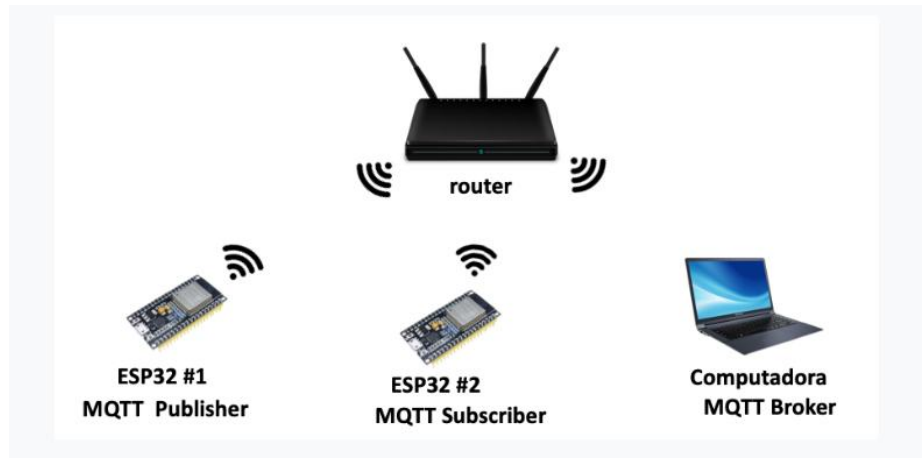
**Garcia Chaves erik**

**Viernes 13 de marzo del 2026**

## Instrucciones

El propósito de la práctica es desarrollar una aplicación IoT con el protocolo MQTT de **SUBSCRIBER/PUBLISHER**

Crear una aplicación el cual un **ESP32** tendrá el rol de **PUBLICADOR** y otro **ESP32** tendrá el rol de **SUBSCRIPTOR** en donde se estará enviando/recibiendo el estado de un led, con la ayuda del bróker **mosquitto**, el tópico al cual se estará enviando será el de **device/led**



Una vez que se haya realizado un ejemplo en un entorno local, sigue utilizar un bróker en un servidor externo.

En el cual se realizará en proceso de publicador o suscriptor de ese bróker externo, el cual se realizará la conexión con las credenciales proporcionadas. Para poder publicar en el tópico **device/led**

Desarrollo:

### Red localhost

Para poder realizar la prueba como localhost es necesario tener corriendo **mosquitto**, para esto en Windows con una terminal de powershell como administrador debemos de correr el siguiente comando:

```
PS C:\Program Files\mosquitto> .\mosquitto.exe -c mosquitto.conf -v
1773378596: mosquitto version 2.1.2 starting
1773378596: Config loaded from mosquitto.conf.
1773378596: Bridge support available.
1773378596: Persistence support available.
1773378596: TLS support available.
1773378596: TLS-PSK support available.
1773378596: Websockets support available.
1773378596: Opening ipv4 listen socket on port 1883.
1773378596: mosquitto version 2.1.2 running
```

Con este comando el bróker esta corriendo correctamente y listo para administrar las suscripciones y publicaciones

### ESP-MQTT subscriber:

El archivo mqtt\_lib.h, contiene un define en donde declaramos la dirección del bróker, en el caso de un localhost seria la dirección IP de nuestra PC que esta haciendo el rol de bróker junto con el puerto al cual se establecerá el enlace, importante dado que MQTT esta sobre TCP.

Después otro define que indica al tópicó al se quiere suscribir, de esta forma el código estará un poco mas limpio

```
#ifndef MQTT_LIB_H
#define MQTT_LIB_H

#define BROKER "mqtt://192.168.1.66:1883"
#define TOPIC "device/led"

void mqtt_event_handler(void *handler_args, esp_event_base_t base, int32_t event_id, void *event_data);

void mqtt_start(void);

#endif
```

### Mqtt\_lib.c

Este es un inicio básico, en el cual la estructura **esp\_mqtt\_client\_config\_t** tiene los miembros necesarios para poder establecer la conexión entre el ESP y el bróker para poder suscribirse a un topic. Indicamos el cliente para poder realizar la conexión.

```
void mqtt_start(void){

    esp_mqtt_client_config_t mqtt_cfg = {
        .broker.address.uri =BROKER,
    };

    esp_mqtt_client_handle_t client = esp_mqtt_client_init(&mqtt_cfg);
    esp_mqtt_client_register_event(client, ESP_EVENT_ANY_ID, mqtt_event_handler, NULL);
    esp_mqtt_client_start(client);

}
```

Lo importante pasa dentro del manejador de eventos **mqtt\_event\_handler** en donde cada evento que se registre en la red, como en este caso que se haya recibido una publicación indicara que se ha publicado entonces recibirá la información y la mostrara por pantalla, en el caso que se desee hacerlo,

```

void mqtt_event_handler(void *handler_args, esp_event_base_t base, int32_t event_id, void *event_data){

    esp_mqtt_event_handle_t event = event_data;
    client = event->client;
    int msg_id;

    switch ((esp_mqtt_event_id_t)event_id){

        case MQTT_EVENT_CONNECTED:{
            ESP_LOGI(TAG, "MQTT_EVENT_CONNECTED");
            msg_id = esp_mqtt_client_subscribe(client, TOPIC, 0);

            ESP_LOGI(TAG, "sent subscribe successful, msg_id=%d", msg_id);
        }break;

        case MQTT_EVENT_DISCONNECTED:{
            ESP_LOGI(TAG, "MQTT_EVENT_DISCONNECTED");

        }break;

        case MQTT_EVENT_DATA:{

            char topic[128] = {0};
            char data[256] = {0};

            snprintf(topic, sizeof(topic), "%.*s", event->topic_len, event->topic);
            snprintf(data, sizeof(data), "%.*s", event->data_len, event->data);

            ESP_LOGI(TAG, "Topic : %s", topic);
            ESP_LOGI(TAG, "Data : %s", data);

            break;

        }break;

        case MQTT_EVENT_ERROR:{
            ESP_LOGE(TAG, "Error MQTT");
        }break;

        default:
            ESP_LOGI(TAG, "Other event id:%d", event->event_id);
            break;

    }

}

```

En este caso los eventos que más queremos monitorear sería cuando se logra conectar al bróker **MQTT\_EVENT\_CONNECTED** y **MQTT\_EVENT\_DATA**. que es el evento donde el cliente, el suscriptor ha recibido un mensaje de un publicador. Este evento contiene información como el **ID del mensaje**, el **nombre del topic**, la **información y la longitud**.

## ESP-MQTT publicador

El código, la estructura para un **MQTT-ESP publicador** es en esencia muy parecido al suscriptor, solo que como su nombre lo indica este estará publicando por lo que no va a tener la carga de estar monitoreando si se ha recibido algo

```
void mqtt_event_handler(void *handler_args, esp_event_base_t base, int32_t event_id, void *event_data){

    esp_mqtt_event_handle_t event = event_data;
    client = event->client;
    int msg_id;

    switch ((esp_mqtt_event_id_t)event_id){

        case MQTT_EVENT_CONNECTED:{
            ESP_LOGI(TAG, "MQTT_EVENT_CONNECTED");
            msg_id = esp_mqtt_client_publish(client, TOPIC, "prueba_publicador", 0, 1, 0);
            ESP_LOGI(TAG, "sent publish successful, msg_id=%d", msg_id);

            }break;

        case MQTT_EVENT_DISCONNECTED:{
            ESP_LOGI(TAG, "MQTT_EVENT_DISCONNECTED");

            }break;

        case MQTT_EVENT_PUBLISHED:{

            ESP_LOGI(TAG, "MQTT_EVENT_PUBLISHED, msg_id=%d", event->msg_id);
            }break;
        case MQTT_EVENT_ERROR:{
            ESP_LOGE(TAG, "Error MQTT");
            }break;

        default:
            ESP_LOGI(TAG, "Other event id:%d", event->event_id);
            break;
    }

}
```

En este manejador de eventos lo único que tendrá será el evento que indica que el bróker ha recibido un **ACK** de la publicación hecha por el cliente.

La manera en cómo realizamos una publicación sería con la función **esp\_mqtt\_cliente\_publish(*cliente, topico, data, length\_data, QoS, retains*)**

En la longitud de la información se agrega 0 y la función calcula en automático el tamaño del payload,

Tenemos un **QoS de 1**, esto quiere decir que el cliente (**ESP**) intentara hasta recibir el **PUBACK** del bróker recibió la publicación.

```
void sent_task(void *params){

    char buffer[40];

    while(1){
        if(xQueueReceive(data_queue, (void*)buffer, portMAX_DELAY)){
            ESP_LOGI(TAG, "recibio cola: %s", buffer);
            esp_mqtt_client_publish(client, TOPIC, buffer, 0, 1, 0);
        }
    }
}
```

## Programa corriendo en local:

Primero se levanta el bróker:

```
erikG Mosquitto 09:47 .\mosquitto.exe -c mosquitto.conf -v
1773422876: mosquitto version 2.1.2 starting
1773422876: Config loaded from mosquitto.conf.
1773422876: Bridge support available.
1773422876: Persistence support available.
1773422876: TLS support available.
1773422876: TLS-PSK support available.
1773422876: Websockets support available.
1773422876: Opening ipv4 listen socket on port 1883.
1773422876: mosquitto version 2.1.2 running
```

El ESP se conecta como suscriptor para el topic “**device/led**”

```
W (774) wifi_lib: reintentando conexion de WIFI.. (intento: 1/ de: 5)
W (3194) wifi_lib: reintentando conexion de WIFI.. (intento: 2/ de: 5)
I (3194) wifi:new:<1,0>, old:<1,0>, ap:<255,255>, sta:<1,0>, prof:1, snd_ch_
cfg:0x0
I (3204) wifi:state: init -> auth (0xb0)
I (3214) wifi:state: auth -> assoc (0x0)
I (3214) wifi:state: assoc -> run (0x10)
I (3234) wifi:connected with INFINITUMF4AF, aid = 93, channel 1, BW20, bssid
= d8:6d:17:90:8f:48
I (3234) wifi:security: WPA2-PSK, phy: bgn, rssi: -42
I (3244) wifi:pm start, type: 1

I (3244) wifi:dp: 1, bi: 102400, li: 3, scale listen interval from 307200 us
to 307200 us
I (3274) wifi:AP's beacon interval = 102400 us, DTIM period = 1
I (6484) esp_netif_handlers: sta ip: 192.168.1.93, mask: 255.255.255.0, gw:
192.168.1.254
I (6484) wifi_lib: IP obtenida: 192.168.1.93
I (6484) wifi_lib: WIFI conectado con éxito!
I (6484) MQTT_LIB: Other event id:7
I (6724) MQTT_LIB: MQTT_EVENT_CONNECTED
I (6724) MQTT_LIB: sent subscribe successful, msg_id=25168
I (6864) MQTT_LIB: Other event id:3
```

El cliente **ESP\_PULICADOR** se encuentra activo y preparado para enviar publicaciones con la detección de un evento en sus GPIO

```
Si : I (796) wifi:dp: 1, bi: 102400, li: 3, scale listen interval from 307200 us
to 307200 us
I (856) wifi:AP's beacon interval = 102400 us, DTIM period = 1
I (3936) esp_netif_handlers: sta ip: 192.168.1.96, mask: 255.255.255.0, gw:
192.168.1.254
I (3936) wifi_lib: IP obtenida: 192.168.1.96
I (3936) wifi_lib: WIFI conectado con exito!
I (3936) MQTT_LIB: Other event id:7
I (3946) main_task: Returned from app_main()
I (3996) MQTT_LIB: MQTT_EVENT_CONNECTED
```

El ESP que se encuentra como suscriptor esta recibiendo el mensaje que el LED se encuentra encendido o apagado mas aparte mi matricula para lleva run registro.

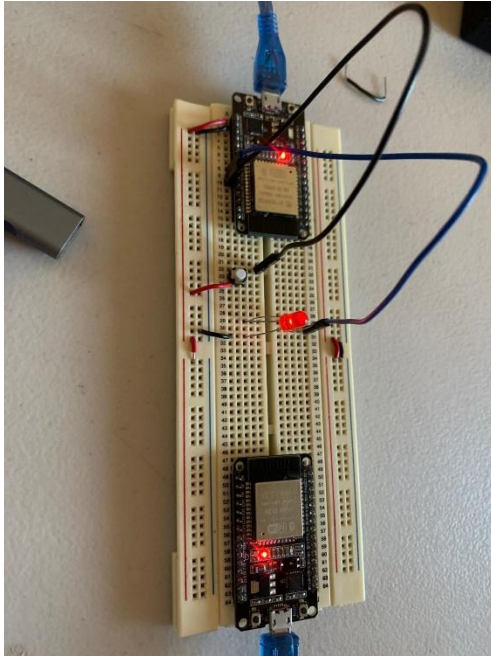
La tarea se encuentra monitoreando un push botton con el que activo el led, mediante un toggle para pasar de OFF – ON / OFF - ON

Por lo que con un consigo sencillo se puede crear una infraestructura realmente compleja incluso mandar comando y reacciones don otros dispositivos IoT.

```
I (176514) MQTT_LIB: Topic : device/led
I (176514) MQTT_LIB: Data : LED ON : 01275863
I (178974) MQTT_LIB: Topic : device/led
I (178974) MQTT_LIB: Data : LED OFF : 01275863
I (180204) MQTT_LIB: Topic : device/led
I (180204) MQTT_LIB: Data : LED ON : 01275863
```

```
I (76506) MAIN: recibio cola: LED ON : 01275863
I (76636) MQTT_LIB: MQTT_EVENT_PUBLISHED, msg_id=34480
I (78966) MAIN: LED OFF
I (78966) MAIN: recibio cola: LED OFF : 01275863
I (79096) MQTT_LIB: MQTT_EVENT_PUBLISHED, msg_id=50292
I (80176) MAIN: LED ON
I (80176) MAIN: recibio cola: LED ON : 01275863
I (80326) MQTT_LIB: MQTT_EVENT_PUBLISHED, msg_id=39903
```

HARDWARE:



### Publicador a un bróker en un servidor

El proceso es de la misma manera, el código para publicar es igual tan solo que las estructura necesita 2 miembros más, el cual es el usuario y la contraseña, ya que el bróker a utilizar es necesario para poder publicar y suscribirse.

```
void mqtt_start(void){

    esp_mqtt_client_config_t mqtt_cfg = {
        .broker.address.uri =BROKER,
        .credentials.username ="mqtt",
        .credentials.authentication.password="mqtt",
    };

    esp_mqtt_client_handle_t client = esp_mqtt_client_init(&mqtt_cfg);
    esp_mqtt_client_register_event(client, ESP_EVENT_ANY_ID, mqtt_event_handler, NULL);
    esp_mqtt_client_start(client);

}
```

Así como cambiar el IP y el puerto dependiendo el puerto si es necesario.

Usaremos una terminal como cliente solo para verificar que se esté publicado correctamente



```
erikG mosquitto_sub ==help to see usage.  
erikG Mosquitto 11:13 .\mosquitto_sub -h 148.231.130.229 -p 50003 -  
u "mqtt" -P "mqtt" -t "device/led"  
LED ON : @1275863  
LED OFF : @1275863  
LED ON : @1275863  
LED ON : @1275863  
LED ON : @1275863  
LED ON : @1275863  
LED ON : @1275863  
LED ON : @1275863  
LED ON : @1275863  
LED OFF : @1275863  
LED ON : @1275863  
LED ON : @1275863  
LED ON : @1275863  
LED OFF : @1275863  
I (6300) MQTT_LIB: Other event id:  
I (6376) main_task: Returned from app_main()  
I (6426) MQTT_LIB: MQTT_EVENT_CONNECTED  
I (11656) MAIN: LED ON  
I (11656) MAIN: recibo cola: LED ON : @1275863  
I (13146) MQTT_LIB: MQTT_EVENT_PUBLISHED, msg_id=39826  
I (15486) MAIN: LED OFF  
I (15486) MAIN: recibo cola: LED OFF : @1275863  
I (15506) wifi:<ba-addr>idx:0 (ifx:0, d8:6d:17:90:Bf:48), tid:0, ssn:5, winSi  
ze:64  
I (15506) MQTT_LIB: MQTT_EVENT_PUBLISHED, msg_id=21315  
I (16806) MAIN: LED ON  
I (16806) MAIN: recibo cola: LED ON : @1275863  
I (24216) MAIN: LED OFF  
I (24216) MAIN: recibo cola: LED OFF : @1275863  
I (27146) MQTT_LIB: MQTT_EVENT_PUBLISHED, msg_id=48176  
I (27676) MQTT_LIB: MQTT_EVENT_PUBLISHED, msg_id=46380
```

Durante la practica se pudo crear una red asíncrona en donde 2 o más dispositivos se pueden comunicar, MQTT es un proco lo ligero creado especialmente para dispositivos de bajos recursos computacionales, en donde para un microcontrolador le queda mas que bien, es un protocolo muy potente, donde un bróker, una computadora central es la que se encarga de distribuir los mensajes de los publicadores a los suscriptores, por lo que un dispositivo, un ESP32 se puede comunicar a otros ESP32, PC's, rasberry pi's, etc., en donde se tenga soporte para esta conexión, no es necesario que estén cercas los dispositivos, pueden que este uno aquí en Tijuana, se comunico con otro en Tecate, por dar un ejemplo, el mensaje llegara de manera eficiente. Por lo que es un protocolo muy potente, ultra ligero, crear aplicaciones IoT es mucho más fácil con este protocolo.

<https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/protocols/mqtt.html>

<https://github.com/espressif/esp-idf/tree/v5.5.3/examples/protocols/mqtt/tcp>